Projet informatique

Gestion des colis et optimisation du chargement d'un camion

Rapport technique

Auteur : [Prénom NOM]

Langage : Python 3

Date : Octobre 2026

Dépôt GitHub : https://github.com/BilalBSK/gestion-colis

Sommaire

1. Introduction et choix du langage
2. Organisation du projet
3. Modélisation des données
4. Gestion des colis : ajout, affichage, recherche
5. Tri des colis
6. Interface utilisateur
7. Passage à l'échelle
8. Optimisation du chargement du camion
9. Prise en compte de la priorité
10. Étude des performances
11. Synthèse des choix algorithmiques majeurs
12. Questions de réflexion
13. Conclusion

1. Introduction et choix du langage

1.1 Contexte

Une entreprise de livraison prépare chaque jour le chargement de ses camions à partir d'une liste de colis. Chaque colis est décrit par un identifiant, un poids, une valeur, une priorité et une destination.

Le programme réalisé doit répondre à deux besoins :

- **gérer les colis** : ajouter, afficher, rechercher et trier une collection pouvant contenir jusqu'à 100 000 colis, voire davantage ;
- **préparer un camion** : choisir les colis à charger sans dépasser la capacité du camion, en transportant la plus grande valeur possible et en favorisant les colis prioritaires.

1.2 Choix du langage : Python

Le projet est réalisé en **Python 3**, avec la bibliothèque standard uniquement. Ce choix repose sur trois raisons :

- **Lisibilité.** Le code Python reste proche du pseudo-code. L'évaluation portant d'abord sur les choix algorithmiques, on peut se concentrer sur les algorithmes plutôt que sur la gestion manuelle de la mémoire (allocation, libération, tailles de tableaux).
- **Outils intégrés.** La bibliothèque standard fournit tout le nécessaire pour tester (`unittest`), mesurer les temps d'exécution (`time`) et la mémoire (`tracemalloc`), et lire des fichiers CSV (`csv`).
- **Structures de données prêtes à l'emploi** (listes dynamiques, dictionnaires), dont le fonctionnement interne et la complexité sont connus et seront expliqués.

Contrepartie assumée : Python est un langage interprété. Une même boucle s'exécute en général 10 à 100 fois plus lentement qu'en C. Cela ne change pas les complexités, mais cela change les temps mesurés. L'étude des performances en tiendra compte.

Respect du sujet (partie 9) : les algorithmes qui constituent le cœur du travail (tris, optimisation du chargement) sont **implémentés à la main**, et leur fonctionnement est expliqué dans ce rapport. Lorsqu'une fonction native est utilisée pour une opération courante, l'algorithme sous-jacent et sa complexité sont précisés. Aucune bibliothèque externe n'est utilisée.

2. Organisation du projet

2.1 Arborescence

Le dépôt respecte l'organisation demandée (dossier `src/`, `README.md`, `rapport.pdf`), complétée par quatre dossiers utiles :

```
projet/
├── src/          code source du programme
├── tests/        tests unitaires
├── donnees/      fichiers de colis (exemple du sujet)
├── docs/         source de ce rapport et captures d'écran
├── outils/       génération du rapport, captures, mesures
├── README.md     présentation et mode d'emploi
└── rapport.pdf   ce rapport
```

2.2 Démarche de travail

Le projet a été réalisé par étapes successives. Chaque étape a été **validée** (tests unitaires et exécution du programme) avant de passer à la suivante, puis enregistrée par un commit sur le dépôt GitHub. L'historique des commits reflète donc la progression du travail.

3. Modélisation des données

3.1 Informations portées par un colis

Le sujet impose au minimum cinq informations par colis. Le tableau suivant précise, pour chacune, le type retenu et les contraintes vérifiées à la création du colis.

Attribut	Type	Contrainte	Justification
identifiant	entier	≥ 0, unique	Sert de clé pour retrouver un colis. L'unicité est vérifiée par la collection (partie 4).
poids	entier (kg)	> 0	Un poids nul ou négatif n'a pas de sens physique. Le choix d'un **entier** est expliqué ci-dessous.
valeur	entier	≥ 0	Une valeur nulle reste possible (colis sans valeur marchande).
priorité	entier	de 1 à 5	Échelle fermée, cohérente avec les exemples du sujet (valeurs de 1 à 5).
destination	texte	non vide	Les espaces en début et en fin sont supprimés (« Paris » saisi avec un espace en trop désigne toujours Paris).

Deux conventions à préciser

- **Sens de la priorité.** Le sujet ne précise pas si 1 ou 5 est la priorité la plus forte. Nous retenons **5 = priorité la plus forte** et 1 = la plus faible, conformément à l'expression « priorité importante » utilisée dans la partie 6 : une valeur élevée correspond à un colis important.
- **Poids entier.** Tous les poids du sujet sont des nombres entiers de kilogrammes. Les conserver sous forme d'entiers est indispensable pour l'algorithme de chargement (partie 8), qui utilise un tableau indexé par la capacité du camion (0, 1, 2, ... kg). Si une précision plus fine était nécessaire, il suffirait d'exprimer les poids en hectogrammes ou en grammes, sans changer l'algorithme.

3.2 Représentation d'un colis en mémoire

Problème	Représenter un colis de façon fiable et compacte, sachant que le programme doit en manipuler jusqu'à 100 000, voire davantage.
Solution retenue	Une **classe de données** Python (`@dataclass`) nommée `Colis`, déclarée avec deux options : • `slots=True` : les cinq attributs sont rangés dans des emplacements fixes de l'objet, au lieu d'un dictionnaire propre à chaque colis ; • `frozen=True` : le colis est **immuable** après sa création.
Complexité en temps	Création et accès à un attribut : O(1).
Complexité en espace	O(1) par colis, soit O(n) pour n colis. Mesure : environ **112 octets par colis**.
Alternatives	Un dictionnaire par colis, une `dataclass` classique (sans `slots`), un tuple nommé (`namedtuple`), ou des tableaux séparés par attribut (un tableau des poids, un tableau des valeurs, etc.).
Justification	**Mémoire :** c'est la représentation la plus compacte parmi celles qui restent lisibles (mesure ci-dessous) : deux fois moins qu'un dictionnaire par colis. **Fiabilité :** l'immuabilité empêche de modifier par erreur l'identifiant d'un colis déjà rangé dans la collection, ce qui rendrait l'index incohérent. **Lisibilité :** on écrit `colis.poids` plutôt que `colis["poids"]` ou `colis[1]`. Les tableaux séparés par attribut seraient encore plus compacts, mais beaucoup moins lisibles. Cette piste n'est intéressante qu'à partir de plusieurs millions de colis (partie 12).

La mesure suivante compare la mémoire nécessaire pour créer 100 000 colis avec chacune des représentations envisagées (script `outils/mesure_memoire_colis.py`, mesure réalisée avec le module `tracemalloc`, liste contenant les colis comprise) :

Représentation	Mémoire pour 100 000 colis	Par colis
Dictionnaire par colis	22,4 Mo	224 octets
`dataclass` sans `slots`	15,2 Mo	152 octets
Tuple nommé (`namedtuple`)	13,6 Mo	136 octets
`dataclass` avec `slots` (retenu)	**11,2 Mo**	**112 octets**

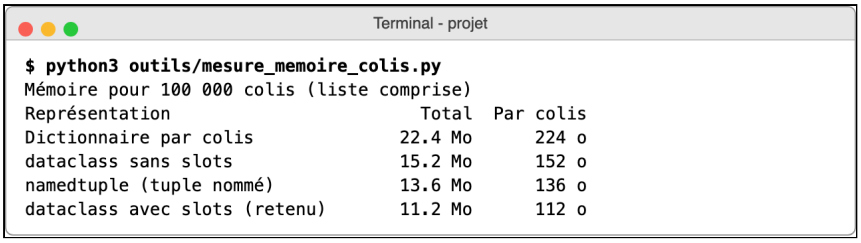


Figure 1 - Mesure de la mémoire occupée selon la représentation choisie

3.3 Validation des données à la création

Chaque colis est vérifié au moment de sa création (méthode `__post_init__`, coût O(1)). Un colis incohérent ne peut donc jamais entrer dans le programme : les algorithmes suivants n'ont pas à se méfier des données. Les règles vérifiées sont :

- identifiant, poids, valeur et priorité sont des **entiers** (un nombre décimal ou un texte est refusé) ;
- poids > 0, valeur ≥ 0, identifiant ≥ 0, priorité comprise entre 1 et 5 ;
- destination non vide.

En cas d'erreur, une exception explicite est levée (par exemple « la priorité doit être comprise entre 1 et 5 »). L'interface pourra ainsi afficher un message clair à l'utilisateur au lieu de s'arrêter.

3.4 Données d'exemple et fichier CSV

Les cinq colis du sujet sont enregistrés dans le fichier `donnees/colis_exemple.csv` :

```
identifiant;poids;valeur;priorite;destination
101;8;20;2;Paris
102;3;10;5;Lille
103;12;40;1;Lyon
104;5;25;4;Paris
105;2;15;3;Lille
```

Le format CSV a été choisi parce qu'il est simple, lisible par un humain et ouvrable dans un tableur. Le séparateur « ; » est celui des tableurs en français, où la virgule sert de séparateur décimal. Le module `stockage.py` lit et écrit ce format, en O(n) en temps et en mémoire pour n colis. Si une ligne est invalide, le message d'erreur indique son **numéro** (par exemple « ligne 3 : le poids doit être strictement positif »), ce qui permet de corriger le fichier facilement.

3.5 Validation de l'étape

Dix-sept tests unitaires (`tests/test_colis.py`) vérifient :

- la création d'un colis valide, y compris aux bornes (priorité 1 et 5, valeur 0) ;
- le refus de chaque donnée invalide : poids nul ou négatif, valeur négative, priorité hors de l'intervalle, identifiant négatif, destination vide, types incorrects ;
- l'immuabilité d'un colis ;
- la lecture exacte des cinq colis du sujet, puis l'écriture et la relecture d'un fichier sans perte ;
- la détection des fichiers mal formés (en-tête incorrect, colonne manquante, valeur non numérique), avec le bon numéro de ligne.

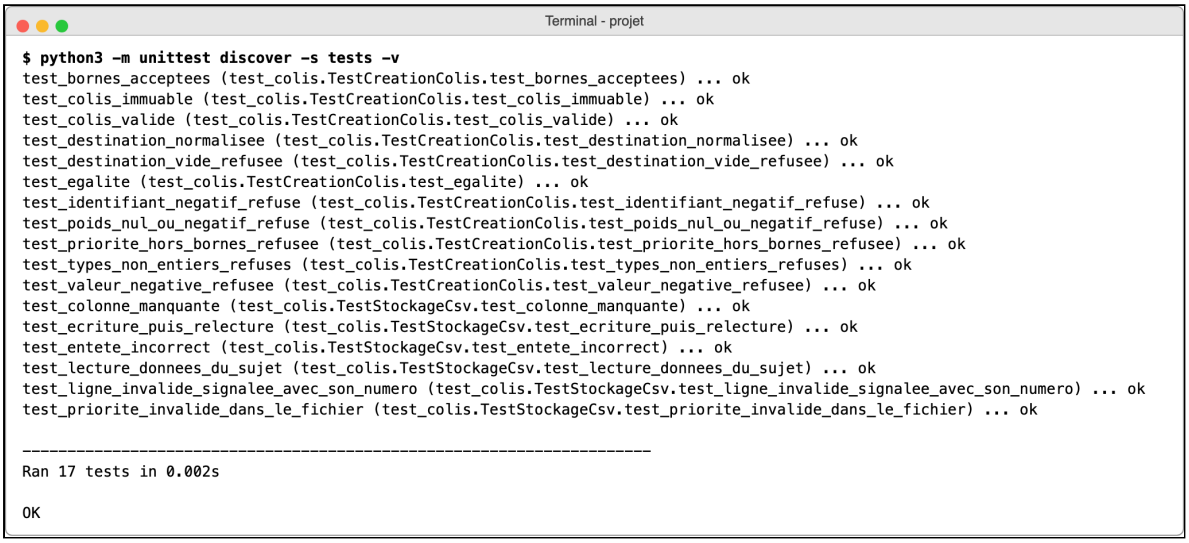


Figure 2 - Exécution des tests unitaires de l'étape 1 : 17 tests réussis

4. Gestion des colis : ajout, affichage, recherche

Cette partie couvre les fonctionnalités 1 à 3 du sujet. Les trois opérations dépendent d'un même choix : la **structure de données** qui contient la collection. Ce choix est donc présenté en premier. Dans toute la suite, **n** désigne le nombre de colis de la collection.

4.1 Structure de la collection : une table de hachage

Structures envisagées

Structure	Ajout (avec contrôle des doublons)	Recherche par identifiant	Parcours	Mémoire
Liste non triée	O(n) : il faut parcourir la liste pour vérifier que l'identifiant est libre	O(n)	O(n), ordre d'insertion	O(n), la plus faible
Liste triée par identifiant	O(n) : il faut décaler les éléments pour insérer au bon endroit	O(log n) par dichotomie	O(n), ordre des identifiants	O(n)
Arbre binaire de recherche équilibré (AVL)	O(log n)	O(log n)	O(n), ordre des identifiants	O(n), mais deux pointeurs par nœud
Table de hachage (retenue)	**O(1) en moyenne**	**O(1) en moyenne**	O(n), ordre d'insertion	O(n)

Fonctionnement d'une table de hachage

Une table de hachage est un tableau de cases. Pour ranger un colis, on calcule à partir de son identifiant un nombre appelé **haché**, qui désigne directement la case où le colis doit être placé. Pour retrouver le colis, on refait le même calcul et on accède directement à la bonne case : **il n'est pas nécessaire de parcourir la collection**.

- **Collisions.** Deux identifiants peuvent désigner la même case. Le dictionnaire Python essaie alors d'autres cases selon une suite déterminée (adressage ouvert). Tant que la table est peu remplie, ces essais supplémentaires restent rares : le coût moyen reste O(1).
- **Agrandissement.** Lorsque la table est remplie aux deux tiers, Python alloue une table plus grande et y recopie tous les éléments. Cette recopie coûte O(n), mais elle est rare, car la taille augmente à chaque fois d'un facteur constant. Réparti sur l'ensemble des ajouts, son coût reste constant : on parle de coût **O(1) amorti**.

- **Pire cas.** Si tous les identifiants tombaient dans la même case, la recherche coûterait O(n). Avec des identifiants entiers, ce cas ne se produit pas en pratique.
- **Ordre.** Depuis Python 3.7, le dictionnaire conserve l'ordre d'insertion : il range les entrées dans un tableau compact, dans l'ordre d'arrivée, et la table de hachage ne contient que leurs positions. On peut donc afficher les colis dans leur ordre d'enregistrement sans liste supplémentaire.

Problème	Stocker un nombre potentiellement important de colis, empêcher deux colis d'avoir le même identifiant et retrouver rapidement un colis à partir de son identifiant.
Solution retenue	Une table de hachage (`dict` Python) qui associe à chaque identifiant le colis correspondant : `identifiant → Colis`.
Complexité en temps	Ajout, test de doublon, recherche : O(1) en moyenne (O(1) amorti pour l'ajout). Parcours complet : O(n).
Complexité en espace	O(n). Mesure : de 30 à 52 octets par colis selon le taux de remplissage de la table, en plus des colis eux-mêmes (partie 3).
Alternatives	Liste non triée, liste triée et recherche dichotomique, arbre binaire de recherche équilibré (tableau ci-dessus).
Justification	La recherche par identifiant et l'ajout sont les opérations les plus fréquentes : la table de hachage les rend **indépendantes du nombre de colis**. Les deux avantages d'un arbre (parcours dans l'ordre des identifiants, recherche par intervalle) ne sont pas demandés par le sujet. Les tris par poids, valeur ou priorité sont traités à part (partie 5). Le surcoût mémoire, environ 5 Mo pour 100 000 colis, est faible.

4.2 Fonctionnalité 1 : ajouter un colis

Problème	Ajouter un nouveau colis à la collection sans jamais créer deux colis de même identifiant.
Algorithme	1. Vérifier que l'identifiant n'est pas déjà une clé de la table (`identifiant in table`). 2. Si c'est le cas, refuser l'ajout avec un message explicite : la collection reste inchangée. 3. Sinon, insérer le colis et mettre à jour le plus grand identifiant connu.
Complexité en temps	O(1) amorti. Charger un fichier de n colis coûte donc O(n).
Complexité en espace	O(1) supplémentaire par ajout.
Alternative	Ranger les colis dans une liste et vérifier les doublons par un parcours : O(n) par ajout, donc $O(n^2)$ pour charger n colis.
Justification	Avec une liste, charger 100 000 colis demanderait environ $n^2/2 = 5$ milliards de comparaisons. Avec la table de hachage, il suffit de 100 000 opérations en temps constant. Le plus grand identifiant est tenu à jour à chaque ajout : l'interface peut ainsi proposer un identifiant libre en O(1), au lieu de rechercher le maximum en O(n) à chaque fois.

4.3 Fonctionnalité 2 : afficher les colis

Problème	Afficher les colis enregistrés de façon lisible, y compris lorsque la collection en contient 100 000.
Algorithme	Parcourir les colis dans l'ordre d'insertion et mettre chacun en forme sur une ligne de tableau (identifiant, poids, valeur, priorité, destination). Un paramètre `limite` permet de n'afficher qu'une page, suivie du nombre de colis restants.
Complexité en temps	O(n) pour tout afficher, O(min(n, limite)) pour une page.
Complexité en espace	O(n) pour la copie de la liste des colis transmise à l'affichage.
Alternative	Toujours tout afficher.
Justification	Afficher tous les colis coûte forcément O(n), puisque chacun doit être écrit. En revanche, 100 000 lignes défilant dans un terminal sont illisibles et prennent plusieurs secondes. Afficher une page à la fois garde l'interface réactive. L'utilisateur pourra tout de même demander l'affichage complet (partie 6).

4.4 Fonctionnalité 3 : rechercher un colis

Problème	Retrouver un colis à partir de son identifiant, ou signaler qu'il n'existe pas.
Algorithme	Accès direct dans la table de hachage : l'identifiant est haché, on lit la case correspondante.
Complexité en temps	O(1) en moyenne (O(n) dans le pire cas théorique).
Complexité en espace	O(1) : aucune mémoire supplémentaire.
Alternatives	**Recherche séquentielle** : comparer l'identifiant à celui de chaque colis, en O(n). **Recherche dichotomique** dans une liste triée par identifiant : on compare l'identifiant cherché à celui du milieu, puis on ne garde que la moitié qui peut le contenir. Coût O(log n), mais la liste doit rester triée : chaque ajout coûte O(n).
Justification	Seule la table de hachage offre à la fois une recherche et un ajout rapides. Les mesures ci-dessous le confirment.

Mesures

Le script `outils/mesure_recherche.py` compare les trois méthodes. Pour chaque taille, il effectue 20 000 recherches d'identifiants existants tirés au hasard (moins pour la séquentielle, trop lente) et donne le temps moyen d'une recherche. La colonne « absent » mesure la séquentielle quand l'identifiant n'existe pas, ce qui est son pire cas. La dernière colonne donne la mémoire occupée par la table de hachage.

n	Hachage (retenu)	Dichotomie	Séquentielle	Séquentielle, id absent	Mémoire de la table
100	23 ns	233 ns	450 ns	824 ns	5 Ko
1 000	25 ns	358 ns	3,5 µs	7,1 µs	37 Ko
10 000	33 ns	476 ns	33,2 µs	68,5 µs	295 Ko
100 000	124 ns	665 ns	518,6 µs	944,3 µs	5 243 Ko

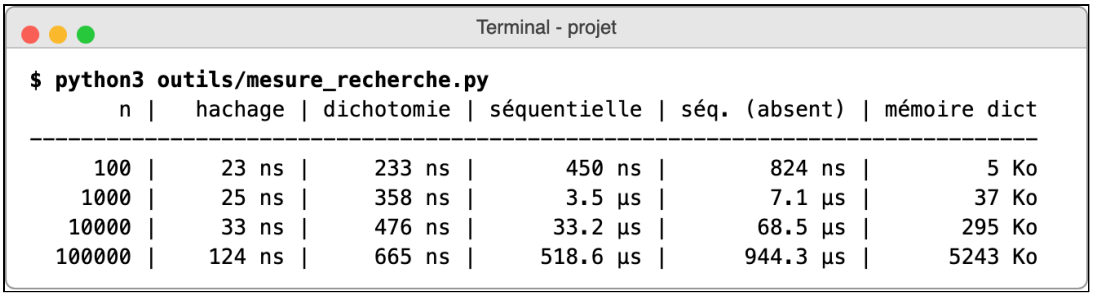


Figure 3 - Temps moyen d'une recherche selon la méthode et le nombre de colis

Analyse des mesures

- **Recherche séquentielle :** le temps est multiplié par environ 10 chaque fois que n est multiplié par 10 (450 ns, puis 3,5 µs, puis 33 µs). C'est le comportement attendu d'un coût O(n). Un identifiant absent coûte environ **deux fois plus** qu'un identifiant présent : en moyenne, on trouve le colis au milieu de la liste (n/2 comparaisons), alors qu'un absent oblige à tout parcourir (n comparaisons). La mesure confirme ce raisonnement.
- **Recherche dichotomique :** le temps n'augmente que d'environ 120 à 190 ns chaque fois que n est multiplié par 10, c'est-à-dire d'environ 3,3 tours de boucle ($\log_2 10 \approx 3{,}3$). C'est la signature d'un coût O(log n).
- **Table de hachage :** le temps reste quasi constant (23 à 33 ns jusqu'à 10 000 colis). À 100 000 colis, il passe à 124 ns : la table occupe alors environ 5 Mo et ne tient plus dans la mémoire cache du processeur, si bien que chaque accès mémoire est plus lent. Le **nombre d'opérations**, lui, ne dépend pas de n. À 100 000 colis, la recherche par hachage est environ **4 000 fois plus rapide** que la recherche séquentielle.
- **Mémoire :** la table occupe entre 30 et 52 octets par colis. Elle augmente par paliers, car la table est agrandie d'un coup lorsqu'elle est remplie aux deux tiers. C'est le prix à payer pour un accès en temps constant.

4.5 Validation de l'étape

Quinze tests unitaires (`tests/test_gestion.py`) vérifient :

- l'ajout d'un colis et le **refus d'un doublon**, la collection restant inchangée, y compris lorsque le doublon provient d'un fichier ;
- le calcul du prochain identifiant libre ;
- la conservation de l'ordre d'insertion, la mise en forme du tableau, l'affichage limité à une page et le cas de la collection vide ;
- la recherche d'un colis existant et d'un colis absent ;
- la **concordance** entre la recherche par hachage et la recherche séquentielle sur 2 000 colis et 202 recherches : les deux méthodes renvoient toujours le même résultat.

Le script `outils/demo_etape2.py` enchaîne les trois fonctionnalités sur les données du sujet :

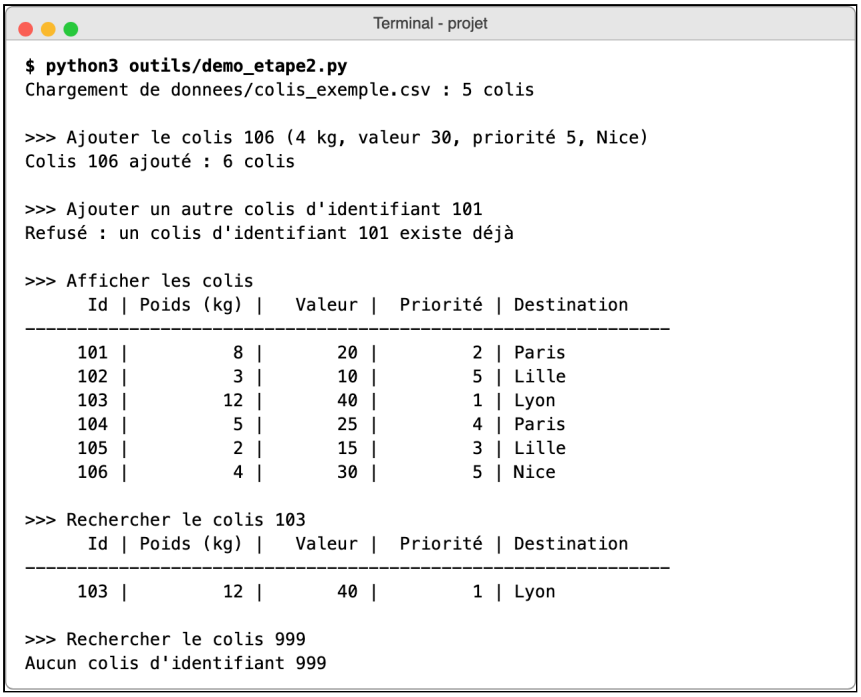


Figure 4 - Ajout d'un colis, refus d'un doublon, affichage et recherche

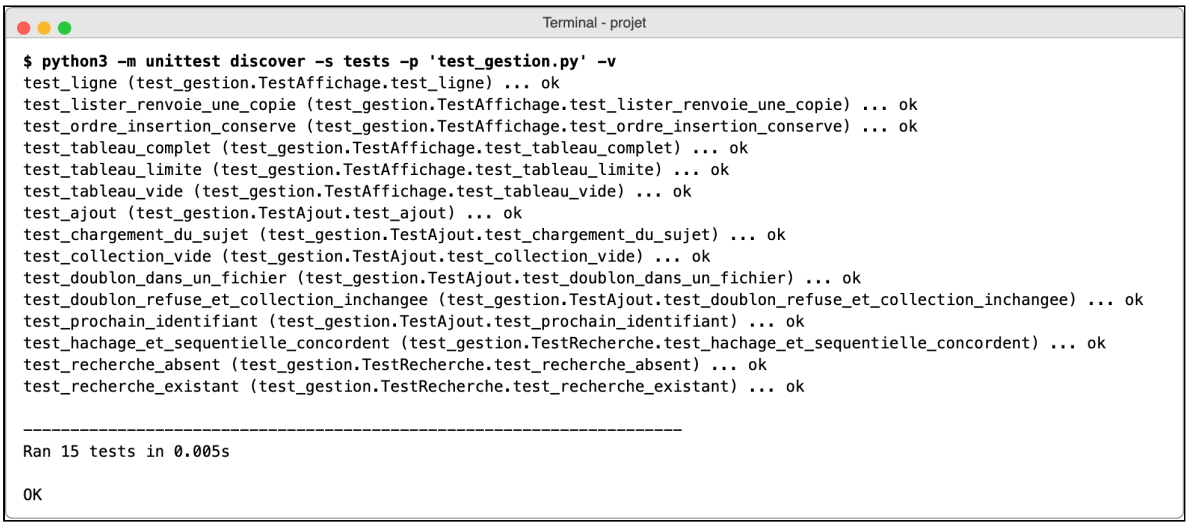


Figure 5 - Exécution des tests unitaires de l'étape 2 : 15 tests réussis

5. Tri des colis

Cette partie couvre les fonctionnalités 4, 5 et 6 du sujet : afficher les colis triés par poids, par valeur et par priorité. Les algorithmes de tri font partie du cœur algorithmique du projet. Conformément à la partie 9 du sujet, ils sont donc **implémentés à la main** (fichier `src/tris.py`) au lieu d'utiliser la fonction native `sorted()`. Celle-ci sert seulement de référence, pour vérifier les résultats et comparer les temps.

Les trois tris ont des caractéristiques communes :

- ils fonctionnent en ordre **croissant ou décroissant** ;
- ils renvoient une **nouvelle liste** : la collection garde l'ordre d'enregistrement des colis, et on peut enchaîner plusieurs tris différents sans rien perdre ;
- ils sont **stables** : deux colis de même clé, par exemple de même poids, restent dans leur ordre d'origine. Cette propriété garantit un affichage prévisible et permet de combiner deux critères (5.3).

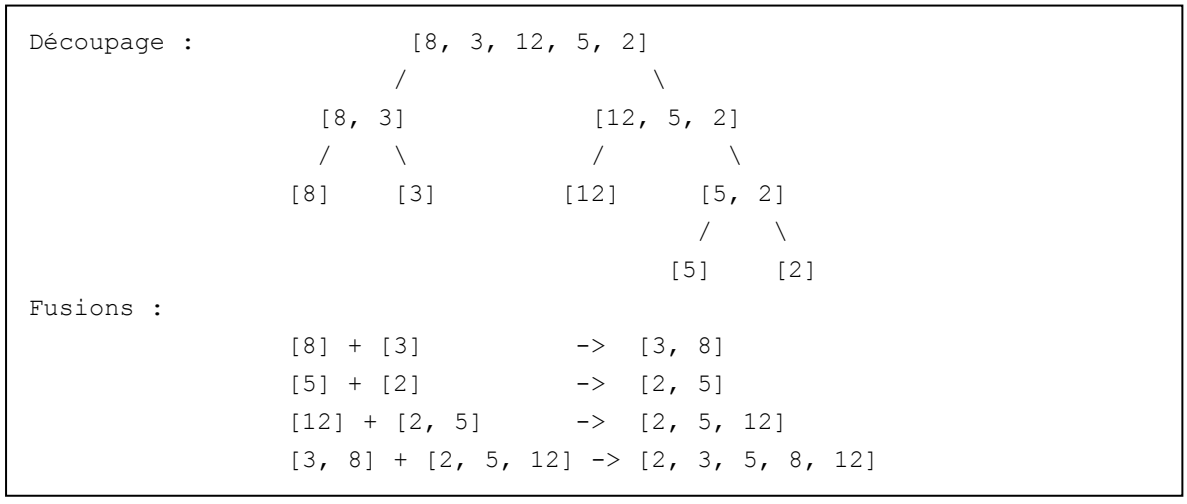
5.1 Tri par poids et par valeur : le tri fusion

Principe

Le tri fusion applique la méthode « **diviser pour régner** » :

1. **Diviser** : couper la liste en deux moitiés ;
2. **Régner** : trier chaque moitié, en appliquant la même méthode récursivement ;
3. **Combiner** : **fusionner** les deux moitiés triées en une seule liste triée.

Une liste de 0 ou 1 élément est déjà triée : c'est la condition d'arrêt. Exemple avec les poids des cinq colis du sujet (101 : 8 kg, 102 : 3 kg, 103 : 12 kg, 104 : 5 kg, 105 : 2 kg) :



La fusion

Pour fusionner deux listes déjà triées, on compare leurs premiers éléments et on retire le plus petit, jusqu'à épuiser l'une des deux listes. Il ne reste alors qu'à recopier la fin de l'autre :

```
Fusion de [3, 8] et [2, 5, 12] :
  3 ou 2  -> 2        résultat : [2]
  3 ou 5  -> 3        résultat : [2, 3]
  8 ou 5  -> 5        résultat : [2, 3, 5]
  8 ou 12 -> 8        résultat : [2, 3, 5, 8]
  il reste [12]       résultat : [2, 3, 5, 8, 12]
```

Chaque comparaison place définitivement un élément : fusionner deux listes de longueurs totales m coûte donc O(m). En cas d'égalité, on prend l'élément de la moitié **gauche**, arrivé le premier : c'est ce qui rend le tri **stable**.

Complexité

La liste est coupée en deux à chaque niveau : il y a donc $\log_2(n)$ niveaux de découpage. À chaque niveau, les fusions traitent en tout les n éléments une fois chacun. Le coût total est donc **O(n log n)**. Ce coût ne dépend pas de l'ordre initial des colis : le tri fusion n'a **pas de pire cas défavorable**. Pour 100 000 colis, cela représente environ 100 000 × 17 ≈ 1,7 million d'opérations élémentaires, contre environ 5 milliards ($n^2/2$) pour un tri naïf.

En mémoire, l'implémentation alloue **une seule fois** deux tampons de n cases (l'un pour les colis, l'autre pour leurs clés), partagés par toutes les fusions, au lieu de créer de nouvelles listes à chaque appel. S'y ajoute la pile des appels récursifs, de profondeur $\log_2(n)$, soit 17 pour 100 000 colis. La clé de tri (poids ou valeur) est lue **une seule fois** par colis, puis déplacée avec lui.

Problème	Afficher les colis classés selon leur poids ou leur valeur. Ces grandeurs n'ont pas de borne fixée à l'avance : un colis peut peser 2 kg comme 800 kg.
Algorithme	Tri fusion récursif, stable, avec des tampons alloués une seule fois (détaillé ci-dessus).
Complexité en temps	**O(n log n)** dans le meilleur, le moyen et le pire cas.
Complexité en espace	**O(n)** : liste des clés et deux tampons de n cases. S'y ajoutent O(log n) pour la récursion.
Alternatives	**Tri par insertion** : simple, mais O(n²). Mesuré ci-dessous, il prend 1 s pour 10 000 colis, et il faudrait environ 100 s pour 100 000. **Tri rapide** (quicksort) : O(n log n) en moyenne, en place, mais O(n²) dans le pire cas et non stable. **Tri par tas** (heapsort) : O(n log n) dans tous les cas et en place, mais non stable. **Tri par comptage** : impossible ici, car le poids et la valeur ne sont pas bornés (voir 5.2). `sorted()` de Python (voir 5.4).
Justification	Le tri fusion est le seul tri par comparaison classique qui soit à la fois **garanti en O(n log n)** et **stable**. La stabilité est indispensable au tri combiné (5.3). Sa contrepartie, la mémoire supplémentaire en O(n), ne représente qu'environ 2,4 Mo pour 100 000 colis (trois listes de 100 000 références de 8 octets).

5.2 Tri par priorité : le tri par comptage

Principe

La priorité ne peut prendre que **5 valeurs** (1 à 5). On peut donc trier **sans comparer les colis entre eux**. On prépare 5 « seaux », un par priorité, et on range chaque colis dans le seau de sa priorité, dans l'ordre où il arrive. Il suffit ensuite de vider les seaux dans l'ordre : de 1 à 5 pour un tri croissant, de 5 à 1 pour un tri décroissant. Exemple avec les colis du sujet et un colis 106 de priorité 5 :

```
Colis parcourus dans l'ordre :
  101 (p2), 102 (p5), 103 (p1), 104 (p4), 105 (p3), 106 (p5)

  seau priorité 1 : [103]
  seau priorité 2 : [101]
  seau priorité 3 : [105]
  seau priorité 4 : [104]
  seau priorité 5 : [102, 106]   <- 102, arrivé avant 106,
                                    reste devant (stabilité)

Lecture des seaux de 5 à 1 (décroissant) : 102, 106, 104, 105, 101, 103
```

Pourquoi est-ce plus rapide qu'un tri fusion ?

On démontre qu'un tri qui procède **uniquement par comparaisons** ne peut pas faire mieux que n log n comparaisons dans le pire cas. Il y a n! ordres possibles des colis, et chaque comparaison apporte une seule réponse oui/non : il faut donc au moins $\log_2(n!) \approx n \log_2 n$ comparaisons pour distinguer tous les cas. Le tri par comptage échappe à cette limite parce qu'il **n'effectue aucune comparaison** : la priorité donne directement le seau. Il parcourt une fois les n colis, puis les k = 5 seaux, d'où un coût **O(n + k) = O(n)**.

Problème	Afficher les colis classés selon leur priorité, une clé entière comprise entre 1 et 5.
Algorithme	Tri par comptage, avec un seau par valeur de priorité (détaillé ci-dessus). Stable.
Complexité en temps	**O(n + k)**, soit O(n) puisque k = 5.
Complexité en espace	**O(n + k)** : les seaux contiennent les n colis et la liste résultat.
Alternative	Le tri fusion de 5.1, en O(n log n), qui fonctionne aussi mais ne tire pas parti du petit nombre de priorités.
Justification	Le domaine de la clé est petit et connu à l'avance : c'est le cas idéal du tri par comptage. Mesuré à 100 000 colis, il prend **2,9 ms, contre 116,5 ms** pour le tri fusion sur la même clé, soit **40 fois plus rapide**. Ce tri ne convient pas au poids ni à la valeur : sans borne connue, il faudrait autant de seaux que de valeurs possibles.

5.3 Tri combiné : priorité, puis valeur

Trier par priorité seule laisse les colis de même priorité dans leur ordre d'enregistrement. Pour les départager par valeur, on exploite la **stabilité** :

1. on trie d'abord par **valeur** (tri fusion) ;
2. puis on trie ce résultat par **priorité** (tri par comptage, stable).

Le second tri regroupe les colis par priorité. Comme il est stable, il conserve à l'intérieur de chaque priorité l'ordre des valeurs obtenu au premier tri. Le coût total est O(n log n) + O(n) = **O(n log n)**. Sur la figure 6, le tri par priorité seul place 102 devant 106 (ordre d'enregistrement), alors que le tri combiné place 106 (valeur 30) devant 102 (valeur 10). Les tests vérifient ce tri sur 1 000 colis aléatoires, en le comparant à un tri de référence sur le couple (priorité, valeur).

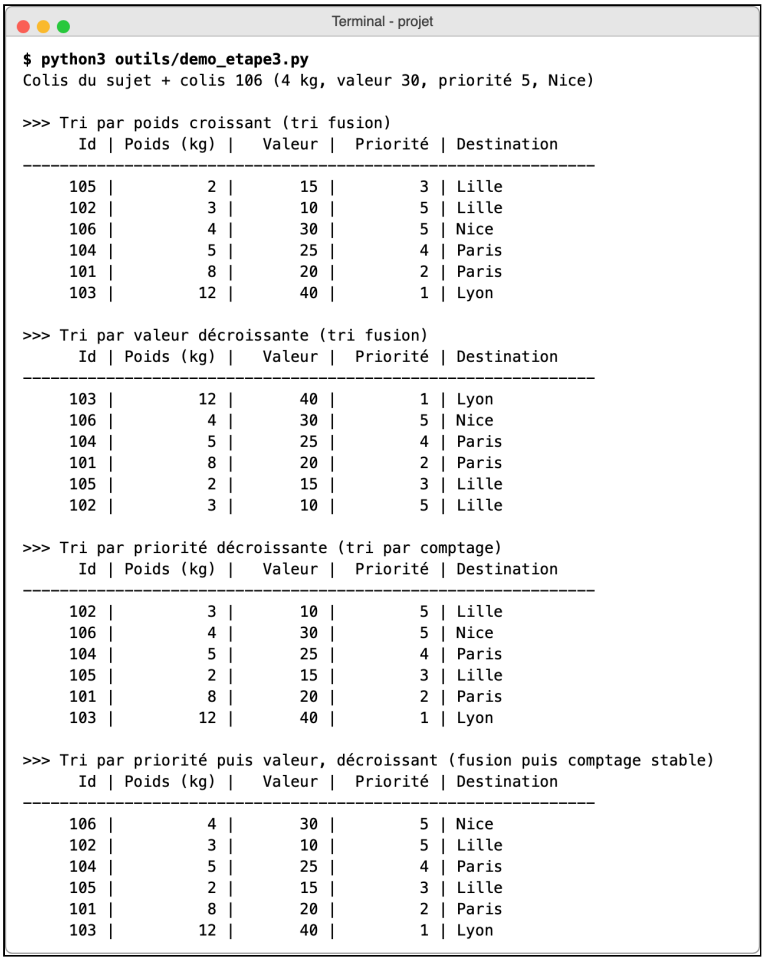


Figure 6 - Les quatre tris appliqués aux colis du sujet (plus le colis 106)

5.4 Fonction native `sorted()` : fonctionnement et raison de ne pas l'utiliser

La fonction `sorted()` de Python utilise l'algorithme **Timsort**. Il repère dans la liste les portions déjà triées (appelées « runs »), trie les petites portions par insertion, puis les fusionne deux à deux comme le tri fusion. Il est stable, s'exécute en O(n log n) dans le pire cas et en O(n) sur une liste déjà triée, et utilise O(n) de mémoire. Comme il est écrit en C, il est environ **19 fois plus rapide** que notre tri fusion écrit en Python (6,2 ms contre 119,9 ms pour 100 000 colis), pour une complexité identique.

Il n'est pas utilisé dans le programme, parce que le sujet demande de maîtriser l'algorithme au cœur de chaque fonctionnalité. Il sert en revanche de **référence** : les tests vérifient que nos tris donnent exactement le même résultat que `sorted()`, y compris l'ordre des égalités. Dans un logiciel en production, on utiliserait `sorted()` pour le poids et la valeur, et l'on conserverait le tri par comptage pour la priorité : même face à `sorted()`, il reste deux fois plus rapide (2,9 ms contre 6,2 ms).

5.5 Mesures

Le script `outils/mesure_tris.py` trie des collections de 100 à 100 000 colis aléatoires et retient le meilleur temps sur plusieurs exécutions. Le tri par insertion n'est mesuré que jusqu'à 10 000 colis : au-delà, il prendrait plusieurs minutes.

n	Poids : insertion	Poids : fusion (retenu)	Poids : `sorted()`	Priorité : comptage (retenu)	Priorité : fusion
100	112 µs	60 µs	4 µs	4 µs	60 µs
1 000	10,6 ms	861 µs	51 µs	34 µs	839 µs
10 000	1,04 s	10,1 ms	595 µs	285 µs	9,8 ms
100 000	≈ 100 s (estimé)	119,9 ms	6,2 ms	2,9 ms	116,5 ms

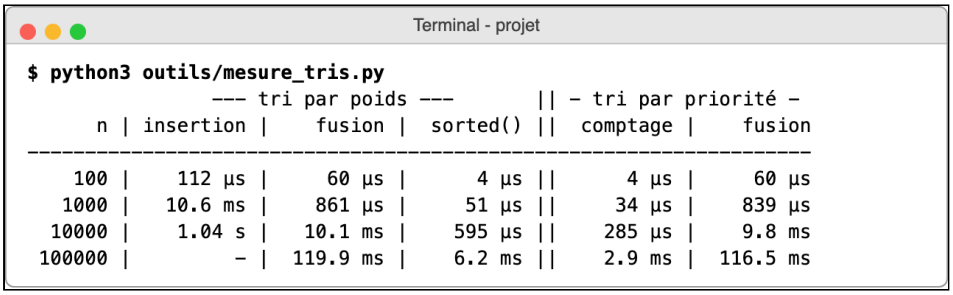


Figure 7 - Temps de tri selon l'algorithme et le nombre de colis

Analyse des mesures

- **Tri par insertion :** le temps est multiplié par environ **100** chaque fois que n est multiplié par 10 (112 µs, puis 10,6 ms, puis 1,04 s). C'est la signature d'un coût $O(n^2)$. À 10 000 colis, il est déjà 100 fois plus lent que le tri fusion.
- **Tri fusion :** le temps est multiplié par environ **12** quand n est multiplié par 10 (10,1 ms, puis 119,9 ms). La théorie prévoit 10 × log(100 000) / log(10 000) = 12,5. La mesure est conforme à O(n log n).
- **Tri par comptage :** le temps est multiplié par environ **10** quand n est multiplié par 10 (285 µs, puis 2,9 ms). C'est bien une croissance linéaire, O(n).

- **Natif contre maison :** `sorted()` a la même croissance que notre tri fusion, avec un facteur constant environ 19 fois plus petit. Ce n'est pas un meilleur algorithme : c'est le même type d'algorithme, exécuté en C plutôt qu'interprété.

Conclusion : avec 100 000 colis, les tris retenus restent largement assez rapides pour une utilisation interactive : 120 ms au plus, 3 ms pour la priorité. Un tri naïf en O(n²) serait inutilisable.

5.6 Validation de l'étape

Seize tests unitaires (`tests/test_tris.py`) vérifient :

- les cas limites : liste vide, un seul élément, liste déjà triée, en ordre inverse, éléments tous égaux ;
- que le tri fusion donne **exactement** le même résultat que `sorted()` sur plus de 50 listes aléatoires, de 0 à 5 000 éléments, dans les deux sens ;
- la **stabilité** des trois tris, sur 2 000 éléments comportant de nombreuses égalités ;
- que la liste d'origine n'est pas modifiée et que la clé n'est calculée qu'une fois par élément ;
- les ordres exacts attendus pour les colis du sujet, pour chaque critère et dans les deux sens ;
- le tri combiné « priorité puis valeur », et la conformité à `sorted()` sur 20 000 colis pour les trois critères.

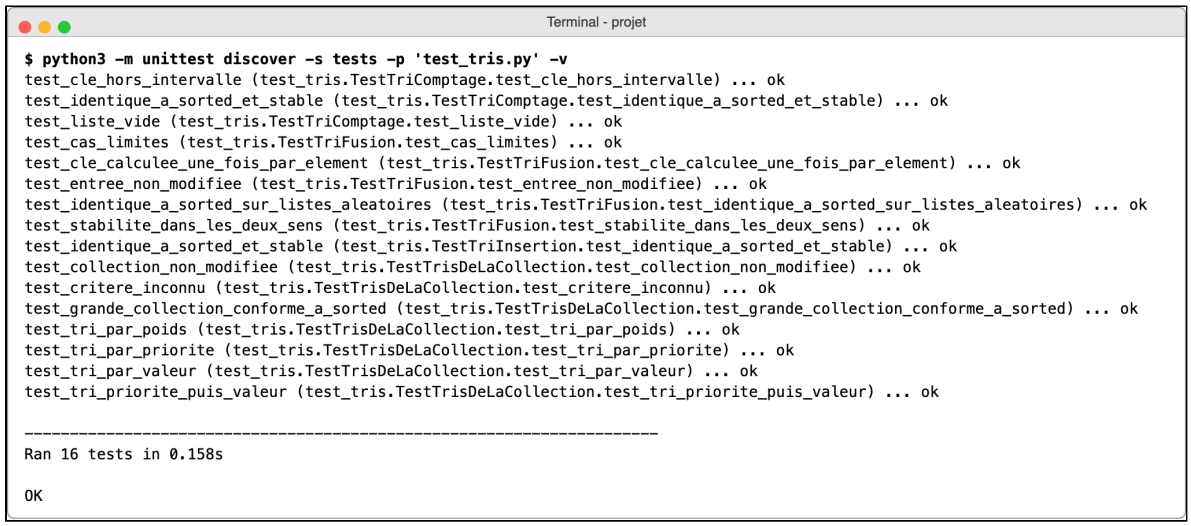


Figure 8 - Exécution des tests unitaires de l'étape 3 : 16 tests réussis